OOP in C++

Operator Overloading

Instructor: Dinesh Maharjan

2022

- What is the difference between an operator & a function?

```
unsigned int Multiply(unsigned x, unsigned y) {
    int prod = 0;
    while (y-- > 0) prod += x;
    return prod;
}

int main() {
    unsigned int a = 2, b = 3;

    // Computed by '*' operator
    unsigned int c = a * b;          // c is 6

    // Computed by Multiply function
    unsigned int d = Multiply(a, b); // d is 6

    return 0;
}
```

Operator	Function

• Usually written in **infix** notation
• Examples:
`Infix: a + b; a ? b : c;`
`Prefix: ++a;`
`Postfix: a++;`
	• Always written in **prefix** notation
	• Examples:
	`Prefix: max(a, b);`
	`qsort(int[], int, int, void (*)(void*, void*));`

• Operates on one or more operands, typically up to 3 (Unary, Binary or Ternary)	• Operates on zero or more arguments
• Produces one result	• Produces up to one result
• Order of operations is decided by precedence and associativity	• Order of application is decided by depth of nesting
• Operators are pre-defined	• Functions can be defined as needed

Operator Overloading

- can specify the meaning of operator whose operands are one or more user-defined types through process known as **operator overloading**
- operators that can be overloaded:

arithmetic	`+ - * / %`
bitwise	`^ & | ~ << >>`
logical	`! && ||`
relational	`< > <= >= == !=`
assignment	`=`
compound assignment	`+= -= *= /= %= ^= &= |= <<= >>=`
increment/decrement	`++ --`
subscript	`[]`
function call	`()`
address, indirection	`& *`
others	`->* , -> new delete`

- not possible to change precedence/associativity or syntax of operators
- meaning of operator specified by operator function, where name of function is **`operator`** followed by operator itself (e.g., **`operator`**+)

Operator Overloading

- Classes can have special member functions to overload built-in operators like +, ==, etc.
- Many overloaded operators can also be written as non-member functions
- Syntax: `return-type operator op (arguments)`
- Overloaded operator functions are selected with the regular overload resolution
- Overloaded operators are not required to have meaningful semantics
- Almost all operators can be overloaded, exceptions are: `::` (scope resolution), `.` (member access), `.*` (member pointer access), `?:` (ternary operator)
- This includes "unusual" operators like: = (assignment), `()` (call), * (dereference), & (address-of), `,` (comma)

Member Pointer Access

```
#include <iostream>
 struct S{
    S(int n): mi(n) {}
    mutable int mi;
    int f(int n) { return mi + n; }};
struct D: public S
{D(int n): S(n) {}};
int main(){
    int S::* pmi = &S::mi; // pointing to member mi by *pmi
    int (S::* pf)(int) = &S::f;
    const S s(7);
//  s.*pmi = 10; // error: cannot modify through mutable
    std::cout << s.*pmi << '\n';
    D d(7); // base pointers work with derived object
    D* pd = &d;
    std::cout << (d.*pf)(7) << ' '
              << (pd->*pf)(8) << '\n';}
```

Operator Overloading (I)

- Mechanism in which we give an additional meaning to existing operators when they are applied to user defined data types e.g. objects
- When an operator is overloaded, its original meanings are not lost
- Improves readability of code and increases scope of operator.

Operator overloading (II)

- General rules of operator overloading-
  - Only existing operators can be overloaded
  - Overloaded operator must have at least one user defined operator
  - Operator function can not have default arguments
  - All binary arithmetic overloaded operator functions explicitly return a value
  - Precedence of operators can not be altered. E.g. * has higher precedence over +

Unary Operator Overloading (I)

- Unary operator acts on single operand(++,--)

- Can be overloaded either through non-static member function or friend function
  - Member function – takes no parameter. E.g. x.operator++()
  - Friend function - takes one parameter. E.g. operator++(x)

- Increment(++) and decrement(--) have two versions, prefix and postfix. To differentiate between them, a dummy parameter of type int is used in postfix

Prefix Overloading

```
#include<iostream>
using namespace std;

class Point{
        int x,y;
        Point(int a, int b){ x=a; y=b;}
        public:
        Point& operator++(){
                x++;
                y++;
                return *this;  // returns reference to current pointer
        }
        void showData(){
                cout<<"x = "<<x<<endl;
                cout<<"y = "<<y<<endl;
        }
};
```

Prefix Overloading

```
int main(){
        Point p(3,4);
        cout<<”During increment”<<endl;
        (++p).showData();
        cout<<”After increment”<<endl;
        p.showData();
}
```

Prefix Overloading with Friend

```
#include<iostream>
using namespace std;

class Point{
        int x,y;
        Point(int a, int b){ x=a; y=b;}
        public:
        friend Point operator++(Point& p);
        void showData(){
                cout<<"x = "<<x<<endl;
                cout<<"y = "<<y<<endl;}};
Point operator++(Point& p){
                p.x++;
                p.y++;
                return p;  // returns object
}
```

Prefix Overloading with Friend

```
int main(){
        Point p(3,4);
        cout<<”During increment”<<endl;
        (++p).showData();
        cout<<”After increment”<<endl;
        p.showData();
}
```

Postfix Overloading

```
#include<iostream>
using namespace std;

class Point{
        int x,y;
        Point(int a, int b){ x=a; y=b;}
        public:
        Point operator++(int){
                Point newPoint=*this;
                x++;
                y++;
                return newPoint;
        }
        void showData(){
                cout<<"x = "<<x<<endl;
                cout<<"y = "<<y<<endl;
        }
```

Postfix Overloading

```
int main(){
        Point p(3,4);
        cout<<"During increment"<<endl;
        (p++).showData();
        cout<<"After increment"<<endl;
        p.showData();
}
```

Postfix Overloading with Friend

```
#include<iostream>
using namespace std;

class Point{
        int x,y;
        Point(int a, int b){ x=a; y=b;}
        public:
        friend Point operator++(Point& p,int);
        void showData(){
                cout<<"x = "<<x<<endl;
                cout<<"y = "<<y<<endl;}};
Point operator++(Point& p,int){
                Point newPoint=p;
                x++;
                y++;
                return newPoint;  // returns object
}
```

Prefix Overloading with Friend

```
int main(){
        Point p(3,4);
        cout<<"During increment"<<endl;
        (p++).showData();
        cout<<"After increment"<<endl;
        p.showData();
}
```

Binary Operator Overloading (I)

- Binary operator is an operator that requires two operands e.g. +,-,=
- Member function –
  - takes one parameter e.g. c.operator+(Circle x).
  - Left hand side operand becomes calling object. R.H.S. becomes passing object.

    e.g. c=c1+c2; -> c = c1.operator+(c2);
  - Left hand operand can not be primary data type as it can not call the function

    E.g. c=100+c1; //error because c=100.operator+(c1) not possible
- Friend function –
  - takes 2 parameters. One parameter has to be user-defined data type. Other can be either secondary or primary data type

    e.g. operator+(Circle c, int n)
  - Both L.H.S and R.H.S. are passed as objects, L.H.S. as 1st parameter and R.H.S. as 2nd parameter

    e.g. c=c1+100; -> c= operator+(c1,100)
  - In case of one of the operands being primary data type, object may appear on either left or right side of operator.

    e.g. C=100+c1; -> c=operator+(100,c1)
- Return type in general is the object of the class

Binary Operator Overloading (II)

- Assignment operators – e.g. =,+=,-=,*= etc
- Assignment operator functions do not return any value. Changes are made in L.H.S. operand
- In case of friend function, first parameter must be an reference to the object
  - e.g. Speed operator+=(Speed &x, Speed y)

    s1+=s2; -> operator+=(s1,s2);
- If an object is assigned to another object at the line of declaration, then copy constructor is called.
  - E.g. Speed s1=s2;
- If it is done on the next line of declaration, then = operator is called.
  - E.g. Speed s1;

    S1=s2;

Binary Overloading

```
#include<iostream>
using namespace std;

class Point{
        int x,y;
        Point(){}
        Point(int a, int b){ x=a; y=b;}
        public:
        Point operator+(Point p1){
                int newx=x+p1.x;
                int newy=y+p1.y;
                return Point(newx,newy);  //anynomous object
        }
        void showData(){
                cout<<"x = "<<x<<endl;
                cout<<"y = "<<y<<endl;}
};
```

Binary Overloading

```
int main(){
        Point p1(3,4);
        Point p2(10,20);
        Point p3=p1+p2;
        p3.showData();
}
```

Binary Overloading with Friend

```
#include<iostream>
using namespace std;

class Point{
        int x,y;
        Point(){}
        Point(int a, int b){ x=a; y=b;}
        public:
        friend Point operator+(Point& p1,Point& p2);
        void showData(){
                cout<<"x = "<<x<<endl;
                cout<<"y = "<<y<<endl;}
};
```

```
Point operator+(Point& p1, Point& p2){
                Point p;
                p.x=p1.x+p2.x;
                p.y=p1.y+p2.y;
                return p;
}

int main(){
        Point p1(3,4);
        Point p2(10,20);
        Point p3=p1+p2;
        p3.showData();
}
```

Binary Overloading with Friend

+ for string

```
#include<iostream>
#include<cstring>

using namespace std;
class String{
    char text[50];
    public:
    String (){}
    String (char a[]){
        strcpy(text,a);
    }
```

+ for string

```
String operator+(String s){
        String s1;
        strcpy(s1.text,text);
        strcat(s1.text,s.text);
        return s1;
    }
    void show(){
        cout<<text<<endl;
    }
};

int main(){
    String s1("Hello "),s2("World"),s3;
    s3=s1+s2;
    s3.show();
    return 0;
}
```

Overloading << and >>

```
#include<iostream>
using namespace std;

class Feet{
        int f;
        public:
        friend void operator>>(istream &fin,Feet& obj);
        friend void operator<<(ostream &fout,Feet& obj);

};
void operator>>(istream &fin,Feet& obj){
                        fin>>obj.f;
}
```

Overloading << and >>

```
void operator<<(ostream &fout,Feet& obj){
                    fout<<obj.f<<endl;
}
int main(){
        Feet obj;
        cin>>obj;
        cout<<obj;
        return 0;
}
```

== Overloading

```
#include<iostream>
#include<string>
using namespace std;

class Money{
        int rupee;
        int paisa;
        public:
        Money(int r,int p){
                        rupee=r;
                        paisa=p;
        }
        bool operator==(Money m);
        void showData(){
                cout<<"Ruppee= "<<rupee<<endl;
                cout<<"Paisa = "<<paisa<<endl;}
};
```

```
bool Money::operator==(Money m){
        if(m.rupee==rupee && m.paisa==paisa)
                return true;
        else
                return false
}

int main(){
        Money m1(1,2), m2(100,200);
        if(m1==m2)
                cout<<"They are same"<<endl;
        else
                cout<<"They are different"<<endl;

        return 0;
}
```

==

Overloading

Type Conversion

- Can be of four types
- Basic (Primitive) to Basic
- Basic to User defined (Class)
- User defined to Basic
- User defined to User Defined

Rules For Type Conversion

- We can convert source type to destination type.
- Source class generally needs operator type casting function and destination class needs parameterized constructor.
- It returns some value of type destination.
- However,if source is primitive then, operator type casting function is not required.
- Similarly, if destination is primitive, then constructor is not required.
- The parameter is of type of source.

Basic to Basic

- Can be implicit (Automatic) or explicit
- If expression has different data types, then the compiler automatically convert low order to higher order type and generate the result.
- If expression has assignment operator then the RHS is calculated and converted to type of LHS.

Basic to Basic

- Automatic conversion (Implicit)

```
int main(){
    int a=10;
    long b=20;
    float c=30.5;
    double d=40.05;
    bool e=true;
    cout<<a+b+c+d+e<<endl; //101.55
    cout<<a/b<<endl; // 0
    cout<<a/c<<endl;  // 0.327869
    a=c;
    d=b;
    cout<<a<<endl; //30
    cout<<d<<endl;  //20
    return 0;
}
```

Basic to Basic

- Explicit Conversion

```
#include<iostream>
using namespace std;

int main(){
    int a=10;
    long b=20;
    float c=30.5;
    double d=40.05;
    bool e=true;
    cout<<float(a/c)<<endl; //0.327869
    cout<<int(a/b)<<endl; // 0
    cout<<int(a+b+c+d+e)<<endl;  // 101
    cout<<char(b)<<endl; // some character
    cout<<int('A')<<endl; // 65
    return 0;
}
```

Basic to User Defined

```
#include<iostream>
using namespace std;
class Distance{
    int meter;
    public:
        Distance(){

        }
        Distance (int m){
            meter=m;
        }
        void display(){
            cout<<"Meter: "<<meter<<endl;
        }
};
```

```
int main(){
    Distance d;
    int meter=6;
    d=meter;
    d.display();
    return 0;
}
```

User Defined to Basic

- Casting operator is used
- Operator keyword is used as casting operator
- operator <data type to be converted to>()
- return value of same type

```
#include<iostream>
using namespace std;
class Distance{
    int meter;
    public:

        Distance (int m){
            meter=m;
        }
        operator int(){
            return meter;
        }
        void display(){
            cout<<"Meter: "<<meter<<endl;
        }
};
```

```
int main(){
    Distance d(5);
    int meter;
    meter=d; //meter=d.operator int();
    cout<<meter<<endl;
    return 0;
}
```

User Defined to User Defined

- Casting operator is used in source class
- Constructor is used in destination class

User Defined to User Defined

```
#include<iostream>
using namespace std;
const float PI=3.14;
class Degree{
    float degree;
    public:
    void setDegree(float d){
        degree =d;
    }
    operator float(){
        return degree*PI/180;
    }
};
```

```
class Radian{
    float radian;
    public:
    void setRadian(float r){
        radian=r;}
    Radian (Degree& d){
        radian=d;}
    void display(){
        cout<<radian<<endl;
    }
};
int main(){
    Degree d;
    d.setDegree(45);
    Radian r=d;
    r.display();
    return 0;
}
```